AIN SHAMS UNIVERSITY • FACULTY OF ENGINEERING

An Intelligent Real-Time Financial Fraud Detection and Risk Assessment Platform

Two-Semester Graduation Project Proposal (2026–2027)

Department: Computer & Systems Engineering

Team Size: 8 Students

Academic Year: 2026 / 2027

Supervisor: Department Faculty Supervisor [Placeholder]

Project Lead / Contact: Team Representation (Mazen Hesham Dewedar & Team)

Submitted in partial fulfillment of the requirements for the Bachelor of Science degree in Computer & Systems Engineering.

1. Executive Summary

Modern digital financial ecosystems face an unprecedented surge in transaction volume and sophisticated cyber threat vectors. Traditional security measures, heavily reliant on static rules or isolated post-transaction reviews, prove inadequate against adaptive fraud rings, account takeovers, and multi-vector synthetic fraud. This proposal outlines the architectural design, engineering methodology, and execution plan for an **Intelligent Real-Time Financial Fraud Detection and Risk Assessment Platform**—a comprehensive enterprise-grade software platform developed by an 8-member team over two academic semesters.

Unlike singular machine learning models or restricted academic scripts, this project is engineered as a full-scale distributed software system. It integrates high-throughput event streaming (Apache Kafka), a robust asynchronous event-driven backend (.NET Core microservices), a specialized Python AI/ML inference service, a polyglot persistence tier (PostgreSQL and Redis), and an intuitive role-based web application (React/Next.js) for end-users, fraud analysts, and system investigators.

The platform ingests real-time transaction streams, extracts behavioral and temporal features, evaluates configurable deterministic business rules, runs high-performance machine learning classification and anomaly detection models, computes a hybrid multi-factor risk score, and dispatches real-time WebSocket alerts to an interactive case-management dashboard. This academic prototype demonstrates how modern software engineering principles, distributed architectures, cybersecurity mechanisms, and AI subsystems can be harmonized to deliver resilient financial monitoring solutions.

2. Introduction

Digital transformation in banking and financial services has revolutionized how individuals and enterprises transact. However, this velocity has introduced severe vulnerabilities. Fraudsters continuously innovate, utilizing automated scripts, credential stuffing, SIM swapping, and coordinated fraud rings to exploit system latency and fragmented visibility. Detecting such malicious activities in milliseconds requires a structural paradigm shift from legacy monolithic batch processing to real-time event-driven architectures.

Static rule-based systems generate excessive false positives while failing to catch zero-day fraud patterns that deviate subtly from known signatures. Conversely, pure black-box artificial intelligence models lack explainability, making them legally and operationally unviable for financial investigators who must justify interventions. This project introduces a hybrid paradigm that couples deterministic rule engines with machine learning classification, unsupervised anomaly detection, behavioral profiling, graph-based entity association, and explainable AI (XAI), underpinned by a rigorous enterprise software foundation.

3. Problem Statement

In modern digital financial platforms, fraudulent activity rarely presents as an isolated, obviously malicious transaction. Instead, fraud manifests as a constellation of subtle behavioral and environmental anomalies distributed across time and space.

> **Illustrative Scenario:** Consider a legitimate user who typically performs transactions around 500–700 EGP from Cairo using a known mobile device during standard daytime hours. Suddenly, a sequence of events occurs:
>
> 1. Multiple login failures occur from an unfamiliar IP address at 3:00 AM.
> 2. A new device fingerprint is registered immediately preceding a password reset.
> 3. A high-value transaction of 30,000 EGP is initiated toward an unverified merchant.
>
> Examined independently, each signal may appear innocuous or insufficient to block service. However, their real-time correlation indicates a high-probability Account Takeover (ATO) followed by fraudulent capital extraction. The academic problem solved by our platform is the automated ingestion, correlation, risk scoring, and investigative presentation of such multi-signal threat vectors in real time.

4. Motivation

This project was selected to address several key engineering and research challenges:

- **Real-World Relevance:** Financial fraud represents a multi-billion-dollar global challenge requiring sophisticated technical countermeasures.
- **Software Engineering Complexity:** Designing a distributed, fault-tolerant, event-driven architecture challenges our mastery of concurrency, asynchronous messaging, and clean microservice boundaries.
- **Cybersecurity Integration:** Embedding defense-in-depth principles—such as RBAC, JWT authentication, rate limiting, and audit logging—simulates real-world enterprise security requirements.
- **AI/ML & Explainability:** Exploring supervised classifiers, anomaly detection algorithms, and SHAP-based model explainability bridges the gap between raw predictive power and actionable human insight.
- **Team Scale Suitability:** An 8-member team provides the ideal human resource capacity to parallelize development across frontend engineering, backend services, event streaming, DevOps infrastructure, AI pipelining, and database design.

5. Project Vision

To design and implement a production-grade, highly scalable, real-time intelligent financial fraud monitoring and investigation platform capable of ingesting high-throughput transaction streams, evaluating multi-factor risk scores through hybrid rules and machine learning, generating explainable alerts, and empowering human investigators with comprehensive case-management workflows.

6. Project Objectives

Project objectives are rigorously categorized across the core engineering and academic domains:

- **Software Engineering Objectives:** Implement a modular, decoupled architecture following SOLID principles and design patterns; construct scalable ASP.NET Core APIs; establish event-driven communication via message brokers.
- **AI/ML Objectives:** Develop supervised fraud classifiers and unsupervised anomaly detectors; implement user behavioral profiling; integrate explainable AI (SHAP/LIME) to clarify risk predictions.
- **Cybersecurity Objectives:** Enforce robust JWT authentication, fine-grained RBAC, rate-limiting, secure password hashing, audit logging, and automated account takeover detection.
- **DevOps Objectives:** Containerize all microservices using Docker; orchestrate local deployments with Docker Compose; establish CI/CD pipelines with automated unit, integration, and security scanning.
- **Frontend Objectives:** Build responsive React/Next.js interfaces featuring real-time WebSocket dashboard updates, transaction monitoring, and case investigation workflows.

7. Scope

In-Scope (Academic Prototype & Simulation)	Out-of-Scope (Production & Real-World Constraints)
• Real-time transaction processing & feature extraction • Hybrid risk engine (Rules + ML + Anomaly + Behavior) • Event-driven architecture (Apache Kafka / RabbitMQ) • Complete case management & investigation workflow • Synthetic transaction generator for demo scenarios • Role-based dashboards (Customer, Analyst, Investigator, Admin)	• Actual banking core system integration • Real financial wire transfers or actual money movement • Handling live, sensitive PII or real customer bank accounts • Production deployment in commercial financial institutions • Automated unilateral blocking without human review options • Direct legal/law enforcement automated integrations

8. Target Users & System Actors

- **Customer:** Views personal accounts, initiates transactions, receives transaction statuses, and reviews alerts concerning their own financial activity.
- **Fraud Analyst:** Monitors transaction streams, reviews system-generated alerts, analyzes risk scores, and flags suspicious transactions for deep investigation.
- **Fraud Investigator:** Creates investigation cases, assigns investigators, reviews collected evidence and audit trails, evaluates related entities, and renders final classifications.
- **Security Officer:** Monitors system security events, analyzes suspicious login attempts, reviews account takeover signals, and audits system access logs.
- **Administrator:** Manages system users and roles, configures global system settings, tunes risk scoring weights, and manages detection rules.

9. Functional Requirements

The platform's functional requirements are systematically defined below:

- **FR-01 User Registration:** Allow new users to register with secure credential verification.
- **FR-02 User Authentication:** Secure login using JWT tokens and multi-factor session tracking.

- **FR-03 Role-Based Authorization:** Enforce strict RBAC permissions across all API endpoints and UI routes.
- **FR-04 Account Management:** Manage user bank accounts, balances, and linked payment instruments.
- **FR-05 Transaction Creation:** Enable users to submit financial transfer and payment requests.
- **FR-06 Transaction Processing:** High-throughput ingestion and publishing of transaction events.
- **FR-07 Transaction Feature Extraction:** Real-time extraction of temporal, velocity, and behavioral features.
- **FR-08 Rule-Based Fraud Detection:** Evaluation of deterministic business rules against incoming transactions.
- **FR-09 ML-Based Fraud Detection:** Inference using supervised machine learning classification models.
- **FR-10 Anomaly Detection:** Unsupervised outlier detection using Isolation Forests or Autoencoders.
- **FR-11 Behavioral Profiling:** Real-time comparison against historical user spending and location baselines.
- **FR-12 Risk Score Calculation:** Aggregation of rule, ML, anomaly, and behavioral scores into a unified risk rating.
- **FR-13 Fraud Alert Generation:** Automatic generation of alerts when risk scores breach configured thresholds.
- **FR-14 Real-Time Alert Delivery:** Instant push of alerts to analysts via WebSockets (SignalR).
- **FR-15 Investigation Case Creation:** Conversion of high-risk alerts into formal investigation cases.
- **FR-16 Case Assignment:** Allocation of cases to specific fraud investigators.
- **FR-17 Evidence Management:** Compilation of related transactions, devices, IPs, and logs into case files.
- **FR-18 Investigation Timeline:** Chronological visualization of user and transaction events during a case review.
- **FR-19 Audit Logging:** Immutable recording of all administrative, analytical, and security actions.
- **FR-20 Security Event Detection:** Real-time detection of brute-force logins and credential stuffing.
- **FR-21 Dashboard Analytics:** Aggregated visual statistics on fraud rates, alert volumes, and resolution times.
- **FR-22 Synthetic Transaction Generator:** Automated generation of normal and fraudulent transaction streams.
- **FR-23 System Monitoring:** Real-time tracking of microservice health, queue lag, and latency metrics.

10. Non-Functional Requirements

- **Performance:** End-to-end transaction evaluation and risk scoring latency under 200ms under standard test load.

- **Scalability:** Horizontal scalability of event consumers and stateless backend microservices via containerization.
- **Availability:** Designed for 99.9% uptime in simulated environments with automatic container restarts.
- **Security:** End-to-end TLS encryption in transit, bcrypt password hashing, JWT security, and parameterized queries protecting against SQL injection.
- **Maintainability:** Clean architecture separation, SOLID adherence, comprehensive unit test coverage (>80%), and modular code structure.
- **Observability:** Structured JSON logging, Prometheus metrics collection, and Grafana performance dashboards.

11. Main Use Cases

Core system operations are modeled through structured use cases:

Use Case ID & Name	Actor	Description & Flow Summary
UC-01: Transaction Processing & Risk Evaluation	Customer / Simulator	Customer initiates a transaction. System publishes event to Kafka, extracts features, queries ML and rule engines, computes risk score, and triggers alerts if thresholds are breached.
UC-02: Alert Review & Triage	Fraud Analyst	Analyst views real-time dashboard, inspects triggered alert details, reviews SHAP explainability factors, and decides whether to dismiss or escalate to an investigation case.
UC-03: Case Investigation & Resolution	Fraud Investigator	Investigator opens assigned case, examines aggregated evidence, timeline, related devices/IPs, adds investigative comments, and sets final status (Fraud, False Positive, Legitimate).
UC-04: Rule & Threshold Configuration	Administrator	Administrator accesses admin panel, modifies risk scoring weights, updates deterministic rule parameters, and manages user roles and permissions.

12. Fraud Scenarios

The platform is architected to detect and analyze seven primary fraud and risk scenarios:

- **12.1 Unusual Transaction Amount:** Deviations where transaction values exceed normal historical spending bounds by configurable statistical margins.
- **12.2 Account Takeover (ATO):** Multi-step attack chains involving credential stuffing, new device registration, rapid profile changes, and immediate fund draining.
- **12.3 Transaction Velocity:** Rapid succession of high-frequency transactions within a compressed time window indicating automated card testing.
- **12.4 Impossible Travel:** Logins or transactions occurring in geographically distant locations within a timeframe rendering physical travel impossible.
- **12.5 Device Anomaly:** Transactions originating from unrecognized hardware fingerprints, modified user agents, or emulated devices.
- **12.6 Suspicious IP:** Traffic originating from known proxy networks, Tor exit nodes, or high-risk VPN pools.
- **12.7 Fraud Rings:** Complex collusion networks sharing common devices, phone numbers, or merchant touchpoints, detected via graph analytics.

13. Overall System Architecture

The platform utilizes a decoupled distributed microservice architecture. Incoming requests from the React/Next.js frontend traverse an API Gateway into the core ASP.NET Core backend services. Transaction events are published to an Apache Kafka event broker, where asynchronous worker services consume events, interact with the Python AI inference service, evaluate rules, compute risk scores, and persist results to PostgreSQL and Redis.

> **[Frontend (React/Next.js)] → [API Gateway / Backend (.NET Core)] → [Event Broker (Kafka)] → [Fraud Detection Engine (Rules + ML + Behavior)] → [Risk Engine] → [Alert & Investigation Service]**

14. Event-Driven Architecture

To decouple high-throughput transaction ingestion from heavy machine learning and rule processing, the system adopts an event-driven architecture powered by Apache Kafka. Producers (Transaction Service) publish immutable transaction events to topic partitions. Dedicated consumer groups process these events asynchronously. This design ensures that spikes in transaction volume do not block user-facing APIs, guaranteeing fault tolerance, horizontal scalability, and decoupled service evolution.

15. Backend Architecture

The primary backend is built using **ASP.NET Core**, adhering to Clean Architecture principles. The solution is divided into distinct layers: API (Controllers/Endpoints), Application (Business logic, DTOs, Validators), Domain (Entities, value objects, domain events), and Infrastructure (Entity Framework Core, database contexts, external service clients). Dependency injection, global exception middleware, and FluentValidation ensure robust, maintainable backend code.

16. AI Service Architecture

Artificial intelligence capabilities are isolated within a dedicated **Python FastAPI microservice**, communicating with the .NET backend via REST/gRPC. This separation ensures that computationally intensive ML inference and model memory footprints do not impact transactional database performance. The AI service leverages scikit-learn, XGBoost, pandas, NumPy, and SHAP for model training, inference, and explainability.

17. AI/ML Pipeline & Supervised Machine Learning

The machine learning lifecycle follows a rigorous pipeline: Raw transaction data ingestion → data cleaning → feature engineering (aggregations, velocity metrics, aggregations) → dataset splitting → model training & cross-validation (Logistic Regression, Random Forest, XGBoost, LightGBM) → evaluation → deployment. Because financial fraud datasets are highly imbalanced, evaluation relies primarily on Precision-Recall AUC (PR-AUC), ROC-AUC, and F1-score rather than raw accuracy.

18. Anomaly Detection & Behavioral Profiling

To capture zero-day fraud patterns where historical fraud labels are absent, the platform integrates unsupervised anomaly detection algorithms including **Isolation Forests** and **Autoencoders**. Simultaneously, the behavioral profiling engine maintains dynamic user baselines (typical spending amounts, frequency, preferred merchants, usual geolocation, and device fingerprints), comparing real-time actions against established historical profiles.

19. Hybrid Risk Engine & Explainable AI (XAI)

The **Hybrid Risk Engine** aggregates outputs from deterministic rules, supervised machine learning classifiers, unsupervised anomaly detectors, and behavioral profiles into a normalized final risk score (0–100):

$$\textit{Final Risk Score} = w_1(\&R_rule) + w_2(\&R_ml) + w_3(\&R_anomaly) + w_4(\&R_behavior)$$

Weights and risk thresholds (Low: 0–30, Medium: 31–60, High: 61–80, Critical: 81–100) are fully configurable by administrators. To ensure regulatory compliance and operational trust, the system integrates **SHAP (SHapley Additive exPlanations)**, providing human investigators with clear, itemized justifications for every risk score (e.g., unusual amount, new device, impossible travel).

20. Cybersecurity Architecture

Defense-in-depth principles are embedded across all tiers: JWT-based stateless authentication, role-based access control (RBAC), bcrypt password hashing with salt, rate-limiting middleware to mitigate brute-force attacks, parameterized SQL queries against injection, automated audit logging of sensitive actions, and real-time brute-force login detection.

21. Investigation and Case Management

When high-risk alerts are triggered, the platform transitions them into formal investigation cases. Investigators can assign cases, review chronological event timelines, inspect related devices, IPs, and accounts, add collaborative notes, and record final resolutions (Fraud, False Positive, Legitimate), maintaining a complete human-in-the-loop oversight mechanism.

22. Graph-Based Fraud Analysis

To uncover sophisticated fraud rings, the platform models entities (Users, Accounts, Transactions, Devices, IPs, Merchants) as a graph data structure. By analyzing shared device fingerprints, IP subnets, and circular transaction flows, the graph analytics engine exposes hidden malicious syndicates that standard relational queries miss.

23. Database Design & API Architecture

The persistence layer utilizes **PostgreSQL** for relational transactional integrity (Users, Accounts, Transactions, Cases, Audit Logs) and **Redis** for caching session tokens, rate-limiting counters, and real-time feature stores. RESTful APIs are structured with standardized resource routing, versioning, pagination, and comprehensive error handling.

24. Frontend Architecture & Real-Time Dashboard

Built using **React and Next.js** with Tailwind CSS, the frontend provides tailored interfaces for Customers, Analysts, Investigators, and Administrators. Real-time updates—such as newly generated fraud alerts and streaming risk scores—are pushed instantly to the analyst dashboard using **ASP.NET Core SignalR (WebSockets)**.

25. DevOps, Monitoring, and Observability

Containerization via **Docker and Docker Compose** ensures environment consistency across development and testing. CI/CD pipelines configured via GitHub Actions automate building, unit testing, integration testing, and container security scanning. System observability is maintained through Prometheus metrics scraping, Grafana dashboards, structured JSON logging, and real-time health checks tracking queue lag and API latency.

26. Dataset Strategy & Synthetic Transaction Generator

The team will leverage established public datasets (e.g., IEEE-CIS Fraud Detection, Kaggle Credit Card Fraud, PaySim) supplemented by a custom-built **Synthetic Transaction Generator**. This simulator models normal user behavior while programmatically injecting complex fraud scenarios (ATO, impossible travel, velocity spikes, fraud rings), enabling robust, repeatable real-time system demonstrations.

27. Testing Strategy

Quality assurance is enforced across multiple testing tiers:

- **Unit Testing:** Backend services, domain logic, and risk calculation algorithms tested via xUnit.
- **Integration Testing:** Database repositories, API endpoints, Kafka event brokers, and AI service communication.
- **End-to-End Testing:** Simulating full customer transaction flows through detection, alerting, and case resolution.
- **Security & ML Testing:** OWASP-based API security testing, rate-limit validation, and rigorous evaluation of ML precision, recall, and false positive rates.

28. Team Member Contributions (8-Member Distribution)

To ensure optimal execution across the two academic semesters, the 8 team members are organized into specialized functional squads:

Squad / Domain	Assigned Members	Core Responsibilities
Backend Architecture & Core APIs	Members 1 & 2	ASP.NET Core microservices, Clean Architecture, EF Core, database migrations, authentication, RBAC.
Event Streaming & Message Broker	Member 3	Apache Kafka cluster setup, topic configuration, producer/ consumer pipelines, event serialization.
AI/ML Engineering & Data Pipeline	Members 4 & 5	Python FastAPI service, feature engineering, ML model training, evaluation, anomaly detection, SHAP integration.
Frontend & Real-Time UI	Member 6	React/Next.js dashboards, Tailwind CSS, SignalR WebSocket integration, data visualization charts.
Cybersecurity, Graph & Case Management	Member 7	Security hardening, audit logging, graph analysis modeling, case management workflow services.
DevOps, Testing & Simulation	Member 8	Docker containerization, CI/CD pipelines, Prometheus/Grafana monitoring, synthetic transaction generator.

29. Two-Semester Implementation Roadmap

Phase	Timeline	Milestones & Deliverables
Semester 1: Foundation & Architecture	Months 1–4	• Requirement analysis & system design specification • Database schema design & core backend microservice setup • Kafka event broker pipeline integration • Initial baseline dataset ingestion & exploratory ML model training
Semester 2: Integration, AI & Evaluation	Months 5–8	• Python AI microservice integration & hybrid risk engine completion • Frontend dashboard, SignalR WebSockets & case management UI • Synthetic transaction generator & end-to-end testing • Performance optimization, security hardening & final documentation